

useRPPG.ts

frontend/hooks | TypeScript

```

/**
 * useRPPG.ts
 *
 * React hook – full rPPG (Remote Photoplethysmography) pipeline.
 *
 * Pipeline:
 * webcam → MediaPipe Face Mesh → forehead ROI (roiDetector.ts) →
 * green channel extraction → Kalman filter → variance score
 *
 * Returns a normalized rPPG variance score (0-100) updated every animation frame.
 * Fed into the weighted fusion model as the "visual" channel (weight: 0.3 standard,
 * drops to 0.1 when signalConfidence < 60%).
 *
 * All processing runs in the browser main thread via requestAnimationFrame.
 * For production, move the pixel extraction loop into a Web Worker to free the
 * main thread – the RPPGExtractor class in rppgExtractor.ts is designed for this.
 */

import { useEffect, useRef, useState, useCallback } from 'react';
import { KalmanFilter, movingAverage, computeNormalizedVariance } from '../utils/kalmanFilter';
import { extractForeheadROI, isHeadPoseAcceptable } from '../signal-processing/roiDetector';

// ■■■ Config ■■■
const BUFFER_SIZE = 150; // ~5 seconds at 30fps
const CALIBRATION_FRAMES = 90; // ~3 seconds for personal baseline
const VARIANCE_WINDOW = 30; // 1-second rolling window for variance
const CONFIDENCE_LOW = 60; // Below this, adaptive weights activate

// ■■■ Types ■■■
interface UserRPPGReturn {
  rppgScore: number; // Normalized variance score 0-100
  isCalibrating: boolean; // True during baseline establishment (~3s)
  faceDetected: boolean; // MediaPipe found a face this frame
  signalConfidence: number; // 0-100, live data quality estimate
  videoRef: React.RefObject;
  canvasRef: React.RefObject;
}

// ■■■ Hook ■■■
export function useRPPG(): UserRPPGReturn {
  const videoRef = useRef<HTMLVideoElement>(null);
  const canvasRef = useRef<HTMLCanvasElement>(null);

  const [rppgScore, setRppgScore] = useState(0);
  const [isCalibrating, setIsCalibrating] = useState(true);
  const [faceDetected, setFaceDetected] = useState(false);
  const [signalConfidence, setSignalConfidence] = useState(0);

  // Signal processing state in refs – avoids stale closure issues in RAF loop
  const signalBuffer = useRef<number[]>([]);
  const baseline = useRef<number>(0);
  const frameCount = useRef<number>(0);
  const kalman = useRef(new KalmanFilter(0.1, 1.0));
  const animFrameId = useRef<number>(0);

  // ■■ Webcam initialization ■■■
  const startCamera = useCallback(async () => {
    try {
      const stream = await navigator.mediaDevices.getUserMedia({
        video: { width: 640, height: 480, frameRate: 30, facingMode: 'user' },
      });
      if (videoRef.current) {
        videoRef.current.srcObject = stream;
        await videoRef.current.play();
      }
    } catch (error) {
      console.error('Error accessing camera:', error);
    }
  }, [videoRef]);

  // Main RAF loop
  useEffect(() => {
    const raf = requestAnimationFrame(loop);
    return () => {
      cancelAnimationFrame(raf);
    };
  }, [startCamera]);

  function loop() {
    if (isCalibrating) {
      // Calibration phase: collect baseline data
      if (frameCount.current < CALIBRATION_FRAMES) {
        if (faceDetected) {
          const greenChannel = extractForeheadROI(videoRef.current);
          signalBuffer.current.push(greenChannel);
          if (signalBuffer.current.length > BUFFER_SIZE) {
            signalBuffer.current.shift();
          }
          frameCount.current++;
        }
      } else {
        // End of calibration
        const avg = movingAverage(signalBuffer.current);
        baseline.current = avg;
        isCalibrating = false;
      }
    } else {
      // Normal operation
      if (faceDetected) {
        const greenChannel = extractForeheadROI(videoRef.current);
        signalBuffer.current.push(greenChannel);
        if (signalBuffer.current.length > BUFFER_SIZE) {
          signalBuffer.current.shift();
        }
        frameCount.current++;

        // Compute variance score
        const variance = computeNormalizedVariance(
          signalBuffer.current,
          baseline.current
        );
        setRppgScore(Math.min(100, Math.max(0, variance)));

        // Update signal confidence
        if (variance < CONFIDENCE_LOW) {
          setSignalConfidence(0.1);
        } else {
          setSignalConfidence(1.0);
        }
      } else {
        setSignalConfidence(0);
      }
    }

    animFrameId.current++;
    requestAnimationFrame(loop);
  }
}

```


